DAY 2 · TERRAFORM ACROSS CLOUDS

One Terraform codebase for 3 clouds? Terraform says no.

~/infra terraform init

```
# "one module for every cloud"
module "network" {
source = "./modules/${var.cloud}"
}

| Error: Variables not allowed
|
| Variables may not be used here.
```

FAILS

TERRAFORM ACROSS CLOUDS

One Terraform project, multiple clouds: what can you actually reuse?

Reusable modules are not the same thing as one universal configuration.

✓ Reuse

Workflow: fmt → validate → plan → review → apply

Module interface: same inputs (name, env, cidr, tags)

Standards: naming, tagging, policy checks

CI pipeline and remote-state pattern

✗ Cloud-specific

Resources: aws_vpc ≠ azurerm_virtual_network ≠ google_compute_network

Identity model and permissions

Network semantics: subnet scope differs per cloud

State backend: S3 / Azure Blob / GCS

```
# Same interface, one implementation per cloud
module "network" {
  source = "./modules/network/aws"  # or /azure, /gcp
  name   = "payments"
  env    = "prod"
  cidr   = "10.20.0.0/16"
}
```

Gotcha: a module's **`source`** must be a literal string. You can't switch clouds with a variable, so keep a separate root configuration per cloud.

01 · THE HOOK

"One Terraform codebase for all three clouds" sounds great until your first resource

There is no universal "network" resource.

`aws_vpc`, `azurerm_virtual_network` and `google_compute_network` are different resources with different arguments and different behavior.

Terraform gives you **one language and one workflow**. It doesn't give you one abstraction over every cloud.

02 · THE PROBLEM

Two traps teams fall into

Trap 1: copy-paste per cloud

Three folders that started identical and quietly drifted apart. Naming, tags and validation differ, and nobody knows which copy is "right".

Trap 2: the universal module

One giant "cloud-agnostic" module packed with count = var.cloud == "aws" ? 1 : 0. It's hard to read, hard to test, and nobody wants to touch it.

The fix: standardize the **interface**, not the implementation.

03 · THE ARCHITECTURE

Shared modules, one root per cloud

```
infra/
├── modules/
│   └── network/
│       ├── aws/      # main.tf variables.tf outputs.tf
│       ├── azure/
│       └── gcp/
└── live/                  # roots: one state each
    ├── aws/prod/network/
    ├── azure/prod/network/
    └── gcp/prod/network/
```

→ **modules/**: one implementation per cloud, same inputs and outputs

→ **live/**: small root configs with the backend, provider and module call

The shared interface

Every cloud's `network` module starts with the same `variables.tf`:

```
variable "name" { type = string }
variable "cidr" { type = string }
variable "tags" {
  type    = map(string)
  default = {}
}

variable "env" {
  type = string
  validation {
    condition     = contains(["dev", "staging", "prod"], var.env)
    error_message = "env must be dev, staging or prod."
  }
}
```

Same names, same validation, same outputs (e.g. `network_id`). Callers learn the interface once.

Different implementations underneath

```hcl
# modules/network/aws/main.tf
resource "aws_vpc" "this" {
  cidr_block = var.cidr
  tags       = merge(var.tags, { Name = "${var.name}-${var.env}" })
}

# modules/network/azure/main.tf
resource "azurerm_virtual_network" "this" {
  name                = "${var.name}-${var.env}"
  address_space       = [var.cidr]
  location            = var.location            # Azure-only input
  resource_group_name = var.resource_group_name # Azure-only input
  tags                = var.tags
}
```

> **Note:** each cloud can add its own extra inputs (Azure needs a location and resource group). In GCP, the CIDR belongs to subnets, not the VPC.

04 · THE GOTCHA

You can't pick the cloud with a variable

```hcl
# ✗ Terraform rejects this: source must be a literal string
module "network" {
  source = "./modules/network/${var.cloud}"
}

# ✓ choose the cloud by folder: one root per cloud
module "network" {
  source = "../../../../modules/network/aws"
  name   = "payments"
  env    = "prod"
  cidr   = "10.10.0.0/16"
}
```

→ **Pin providers** in `required_providers` and commit `.terraform.lock.hcl`

→ **Version shared modules** with Git tags, e.g. `?ref=v1.4.0`

05 · THE TRADE-OFFS

Thin per-cloud modules vs one universal module

Per-cloud modules

✓ Readable and testable
✓ Use each cloud's strengths
✗ Some repeated code
✗ Keeping interfaces in sync takes discipline

Universal module

✓ One entry point
✗ Conditionals everywhere
✗ Lowest common denominator
✗ Every change risks every cloud

Rule of thumb: some duplication is cheaper than the wrong abstraction.

06 · THE TAKEAWAY

Multi-cloud Terraform checklist

- ☐ Same workflow everywhere: fmt → validate → plan → review → apply
- ☐ One module interface per capability, one implementation per cloud
- ☐ Input validation on shared variables
- ☐ One root config (and state) per cloud, env and component
- ☐ Providers pinned, lock file committed
- ☐ Shared modules versioned and tagged
- ☐ One naming and tagging standard, enforced in CI

FURTHER READING

Go deeper: official references

HashiCorp: Terraform modules
`developer.hashicorp.com/terraform/language/modules`

HashiCorp: Module sources
`developer.hashicorp.com/terraform/language/modules/sources`

HashiCorp: Dependency lock file
`developer.hashicorp.com/terraform/language/files/dependency-lock`

Docs change. Always check the current version before you design.

07 · YOUR TURN

Would you use one repository or separate repositories for each cloud?

Tell me your setup in the comments, and what made you choose it.

Follow Deji Ajishe

for the rest of this 30-part multi-cloud DevOps series

♻️ Repost or share this with an engineer on your team

linkedin.com/in/deji-ajishe-64a35722a